

Co-design Group 25214

“PinkPong”

Notes

Uliana Kozlova, Mikhail Berdyugin & Anna Litasova

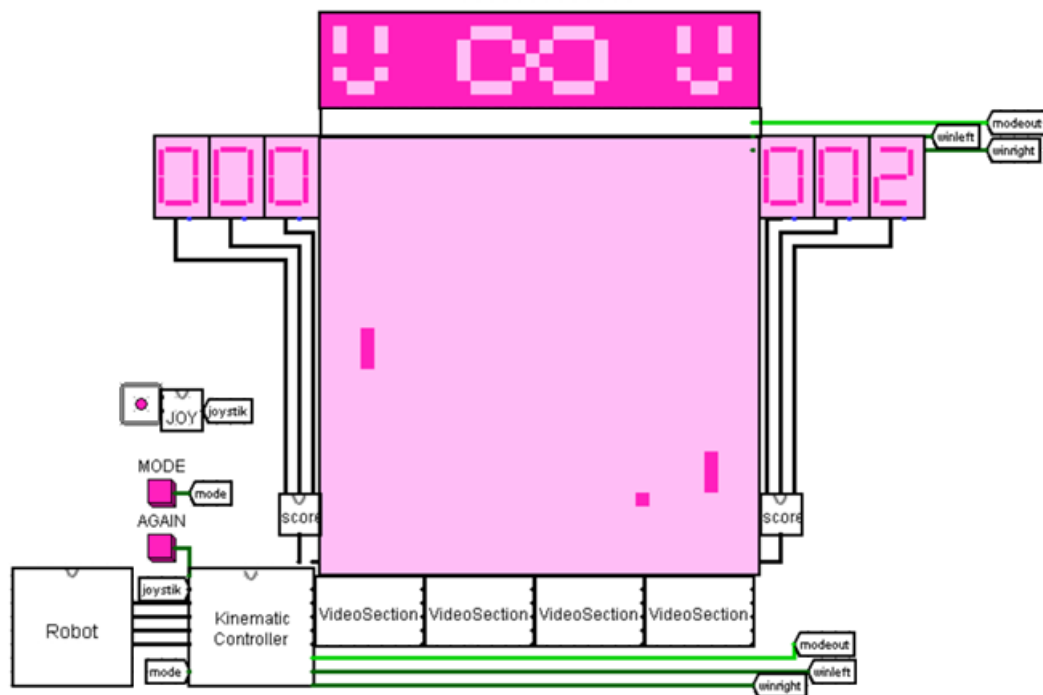
2026

Content

1. **Introduction**
 - 1.1. Project Description
 - 1.2. Additions Beyond the Specification
2. **Description of Additions to the Hardware**
 - 2.1. Subcircuits for the Kinematic Controller and Counters
 - 2.2. Ball Spin
 - 2.3. Ball Diffusion and the Effect of Racket Position on Vertical Ball Speed
 - 2.4. Game Modes
 - 2.5. Win/Fail Display Screen
3. **Description of the Robot's Operation and Its Connection to the Hardware**
 - 3.1. Connecting the Robot to the Hardware
 - 3.2. Software Part of the Project

INTRODUCTION

Project Description



(The project is an implementation of the "TV Tennis" game for one player using Logisim for the hardware part and a CdM-8 processor for the second player, based on the documentation "4COM1042 [Computing Platforms] Co-design Group Project B "The game of TV-Tennis"', written by Alex Shafarenko and Steve Hunt in 2016)

Additions:

- Ball spin relative to the horizontal axis based on bat movement up/down
- Ball diffusion (random change in vertical speed when hitting the floor/ceiling)
- Hitting different pixels of the bat affects the ball's vertical speed
- Two games mode
- Screen displaying the game mode and win/loss
- Smooth bats movement
- The robot takes into account in its calculations the change in vertical speed upon bounce from -4 to 3, rather than to 4

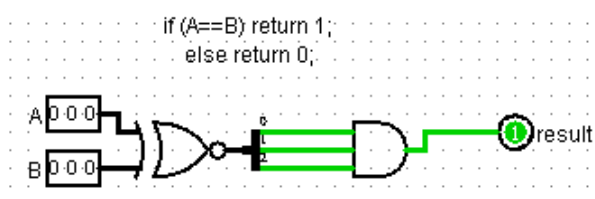
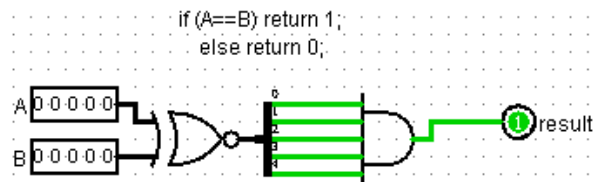
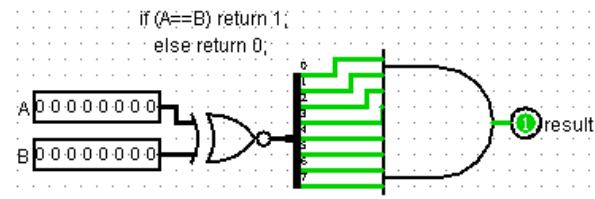
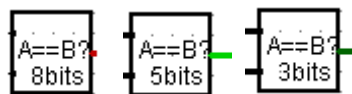
DESCRIPTION OF ADDITIONS TO THE HARDWARE

Subcircuits

1) Equal8Bits, Equal5Bits, Equal3Bits

These blocks are equality comparators of different bit widths (8, 5, and 3 bits). They compare numbers A and B bitwise using an XOR element: if all corresponding bits match completely, 1 is output to result. In any other case, the output will be 0.

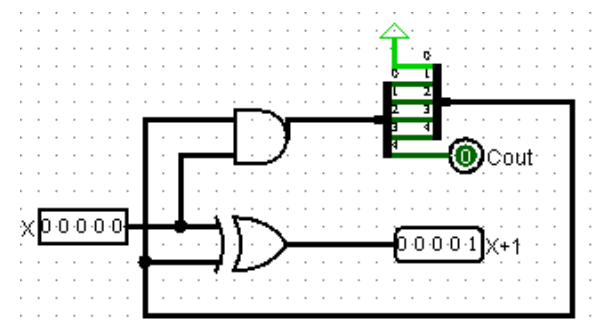
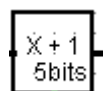
Appearance of the subcircuits:



2) Inc5bits

This block adds one to the input value of 5-bit X. The XOR element computes the new bit value, and the AND generates the carry into the next digit. The result is output to X+1, and the overflow signal is output to Cout.

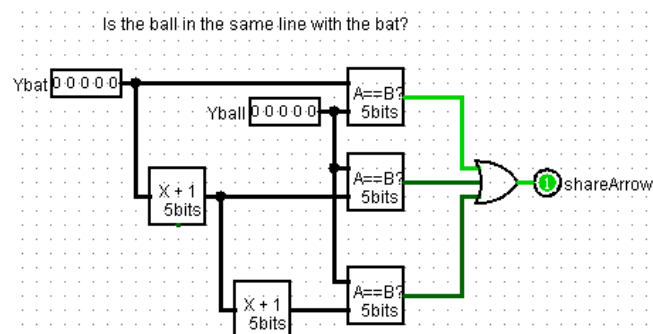
Appearance of the subcircuit:



3) Inrange

This block compares the vertical position of the ball (Y_{ball}) with the vertical position of the bat (Y_{bat}) using its own bit-wise comparators. The result of the comparison is a single shared signal, `shareArrow`, which determines whether the coordinate of at least one pixel of the bat matches the pixel of the ball or not.

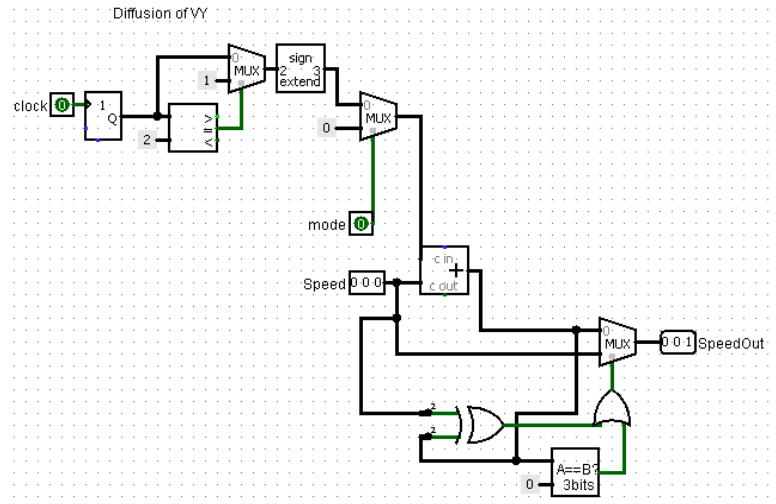
Appearance of the subcircuit:



4) Diffusion

This block uses a 2-bit random number generator to introduce changes to the ball's vertical speed in normal mode (0). This makes the bounce angle less predictable, and the multiplexers ensure that the resulting speed at the SpeedOut output does not exceed the allowed limit and does not become equal to 0.

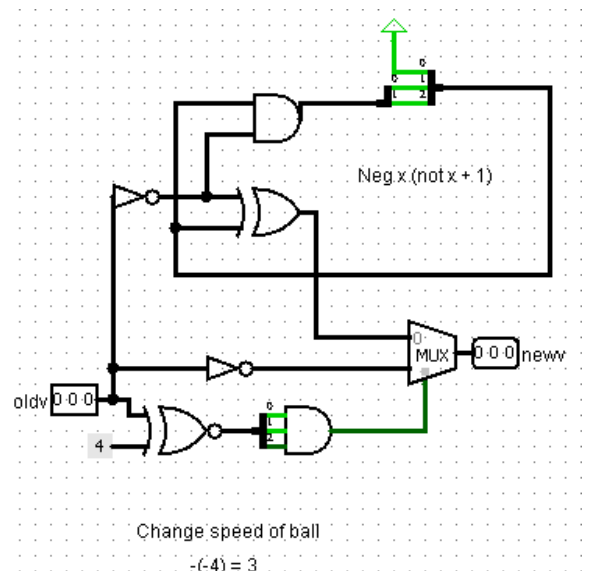
Appearance of the subcircuit:



5) Reflect

This block is responsible for the physics of the ball's bounce by inverting its speed. It takes the current speed value (`oldv`), and changes the sign of the number to the opposite. The multiplexer corrects the case where $v_y = (-4)$, so that this speed changes to 3, and loads the result into `newv`.

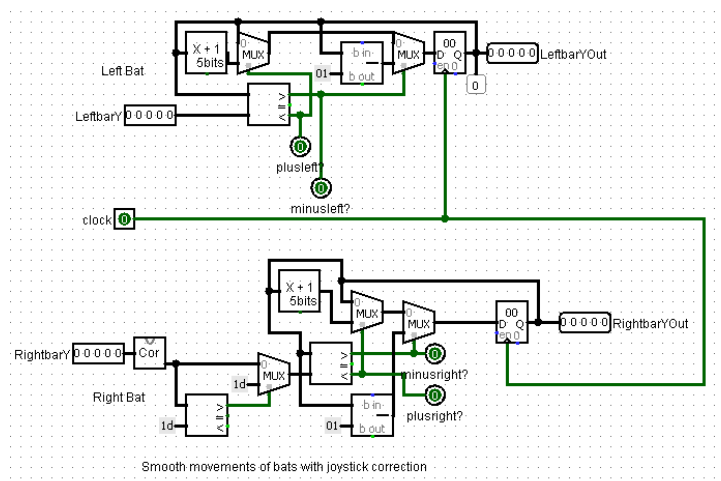
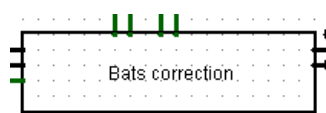
Appearance of the subcircuit:



6) Bats

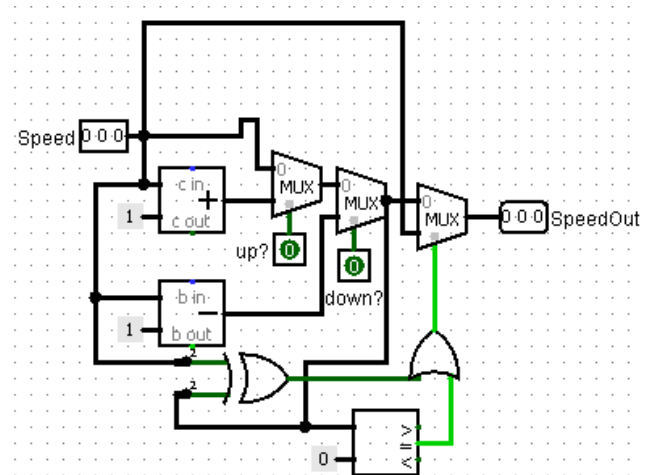
This block controls the smooth vertical movement of the left and right bats. The circuit compares the current bat value with the new one, and depending on this, sets the bat's "speed" - 1, 0, -1. It then adds the bat's speed to its previous position. This ensures that the bats move by exactly 1 pixel per 1 clock cycle. It also passes the speed values (if any) to the kinematic controller for ball spin.

Appearance of the subcircuit:



7) BatReflectVy

This block calculates the updated value of the ball's vertical speed depending on the zone of its impact with the bat. It takes the current speed (Speed) and, if the up? or down? signals are active, its value is increased or decreased by one using adders, changing the bounce angle. If the ball hits the center, the original speed passes through the multiplexer system unchanged. The final result is output to SpeedOut via multiplexers.

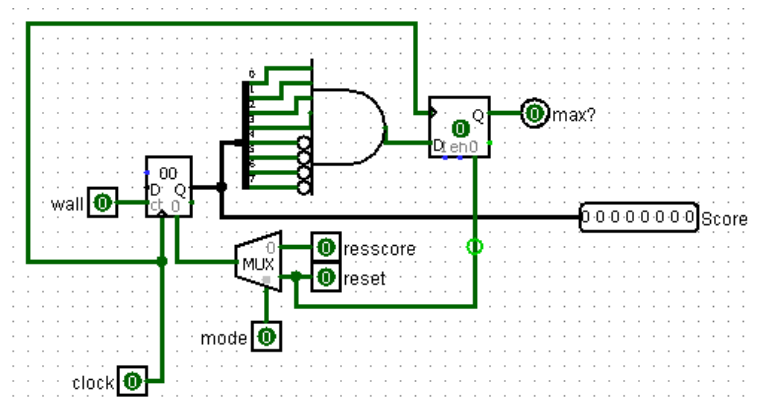


Appearance of the subcircuit:

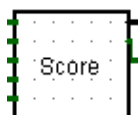


8) Score

This block is responsible for score counting and determining the end of the game. When the ball collides with a wall (wall), the value at the Score output increases, and when it reaches the maximum, the AND element activates the win signal max?. Score reset is implemented via a multiplexer using the reset or resscore signals, which provide reset for different game modes.

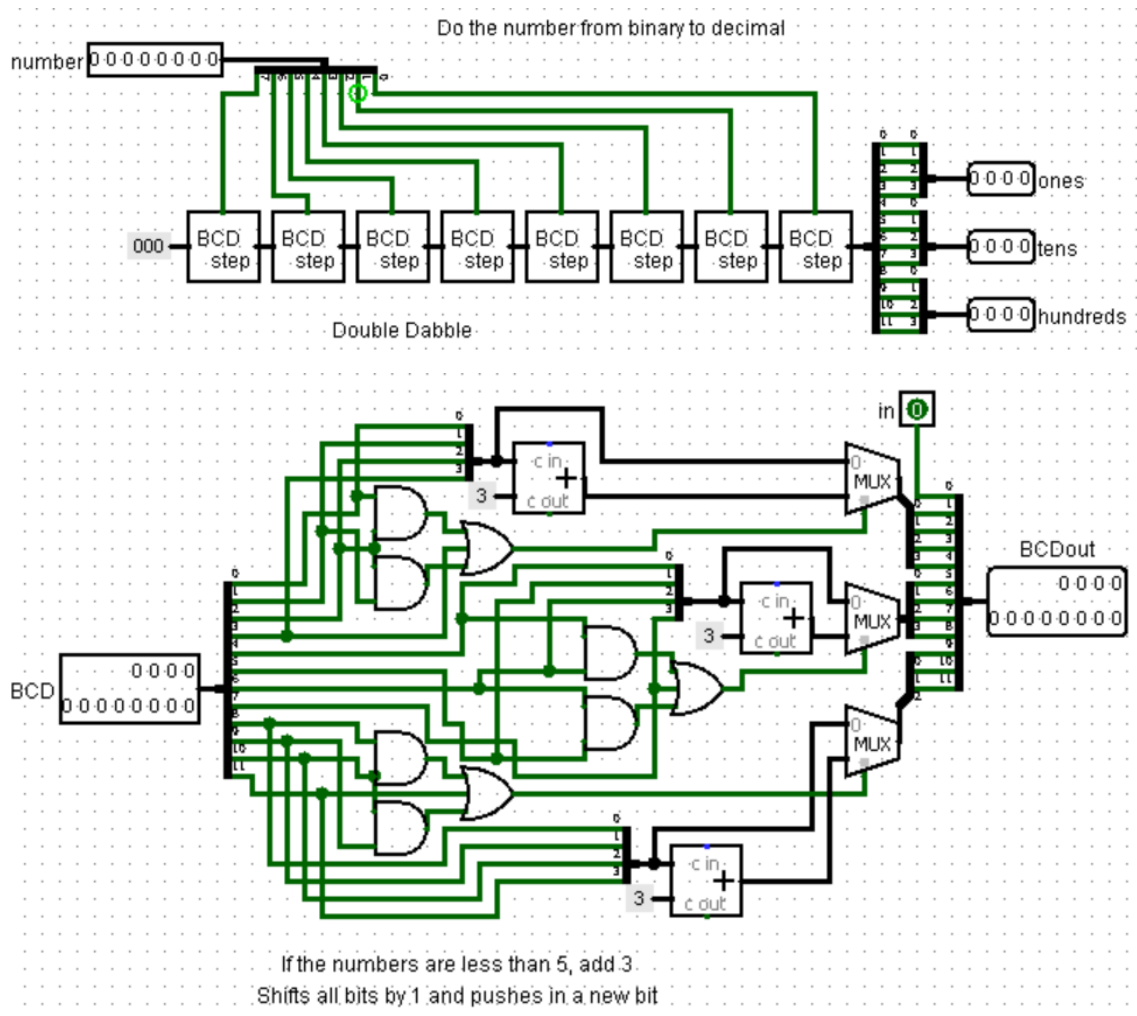


Appearance of the subcircuit:



9) Binaryto3Dig and BCDstep

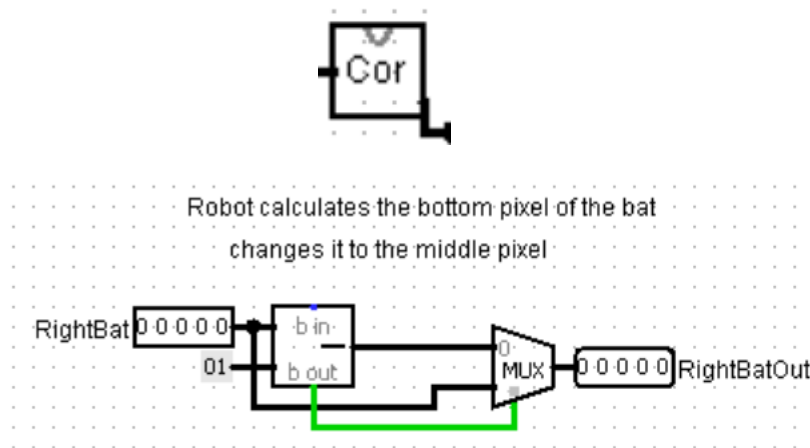
Converts an 8-bit number into 3 4-bit digits for displaying the score on hexadecimal indicators (but in the decimal number system). Each BCDstep checks whether each digit is greater than or equal to 5 and in this case adds 3, since otherwise the digit would become greater than or equal to 10 after the shift, which does not correspond to the decimal number system. It then shifts the number obtained in the previous step by 1, shifting in a new bit of the number. Binaryto3Dig contains a chain of 8 BCDsteps, with each one shifting in a new bit of the original number. Appearance of the subcircuit:



10) CorrectRightBat

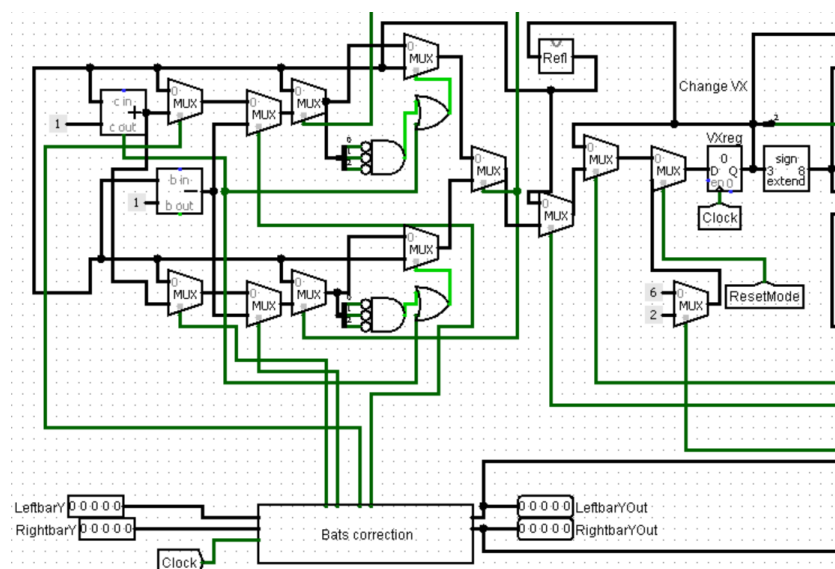
Corrects the position of the robot's bat. The bat is displayed relative to the top pixel, so the robot calculates exactly that one, and to reduce calculation error, the circuit shifts the bat down by 1 pixel (in this case, the pixel calculated by the robot ends up in the middle of the bat).

Appearance of the subcircuit:



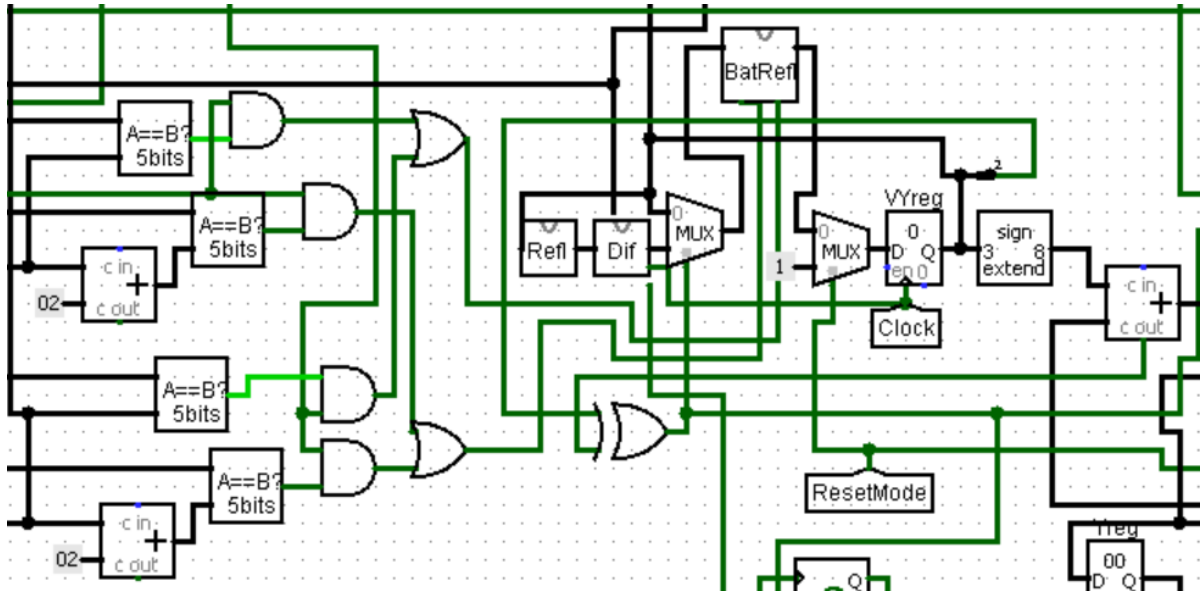
Ball spin

The block is responsible for changing the horizontal speed. The multiplexers process collision signals, calculating how the ball's speed should change upon impact using the Reflect subcircuit. The circuit also takes into account the bat speed signals coming from the Bats subcircuit. If a bat hit occurs and the bat was moving up at that moment, the ball's speed increases by one. If it was moving down, it decreases by one. It also checks that the sum/difference does not change the sign of the speed and that the speed does not become equal to 0; otherwise, the speed remains unchanged.



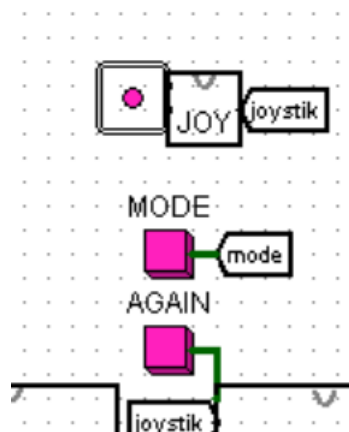
Ball diffusion and the influence of bat position on the ball's vertical speed

The block is responsible for changing the vertical speed. In the event of a bounce off the wall or floor, it changes the speed to the opposite using the Reflect subcircuit, then engages diffusion. It checks whether the ball has hit the top or bottom pixel of the bat; in this case, it changes the speed using the BatReflectVy subcircuit.



Game modes

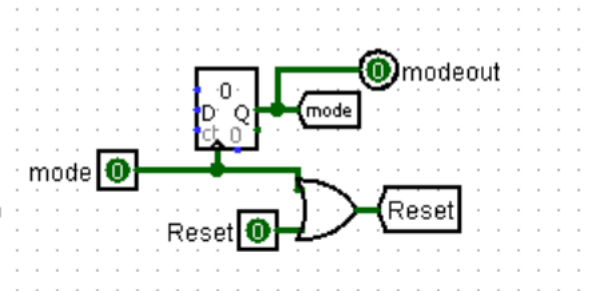
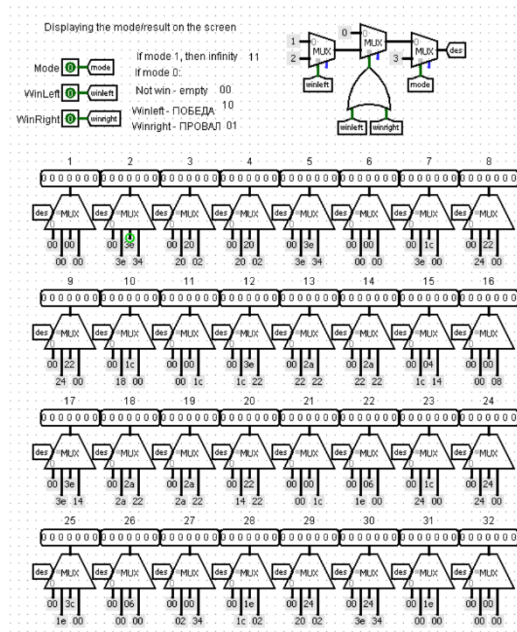
As mentioned earlier, the game has two modes. Their main difference is that in the second mode – let's call it "infinite" – there is no way to beat the robot; it will always correctly predict the ball's trajectory. In the classic mode, however, there is diffusion when hitting the ceiling and floor, due to which the ball can change its trajectory in the opponent's half of the field – at that moment, the robot no longer calculates the new position. In addition, in infinite mode, the counters work up to 255 points, and when a new point is reached after 255, they simply start over from zero. In classic mode, the maximum is 15 points, followed by a win or loss, upon reaching which the game cannot be continued without switching the mode or pressing the AGAIN button. Otherwise, including in the robot's logic, the modes do not differ.



Screen displaying win / loss

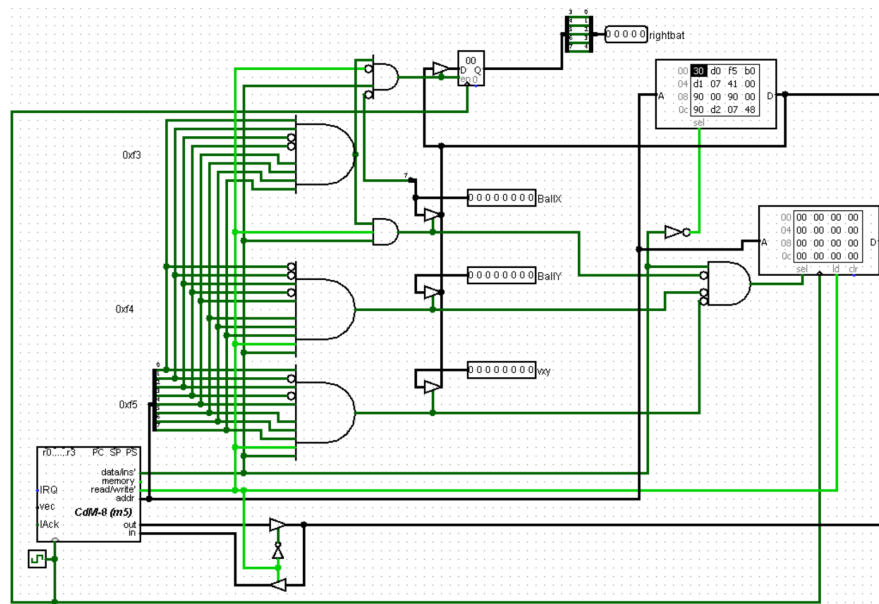
On the main screen, the player has a button that, when pressed, changes the mode, resets the counters, and starts the game over in the new mode. In the kinematic controller, this is implemented using a counter whose clock input is replaced by the button input. Also, this switch is connected to Reset so that the game starts over when a new mode is selected.

The chip itself, which displays the mode or loss/win on the screen, consists primarily of multiplexers that supply the required constant depending on the conditions input from outside.



DESCRIPTION OF ROBOT'S OPERATION AND ITS CONNECTION TO THE HARDWARE

Connecting the robot to the hardware



The robot is built on the CdM-8 processor, which interacts with the hardware part via I/O ports. The robot's program is stored in ROM. A stack in RAM is used for temporary data storage.

The processor reads the ball's coordinates (BallX, BallY) and its speed (vxy) at addresses 0xF3–0xF5, and also controls the right bat (rightbat) via port 0xF3. To pick the right port, there's a circuit that compares the address against a constant using AND gates. The kinematic controller runs on a clock that's 14 times slower than the processor's, so the robot has time to finish its calculations.

The software

The robot's program runs in an infinite loop using an unconditional branch, and the processor runs without a hardware reset to keep performance as high as possible. Each time through the loop, the robot works out the new bat position with the formula $Y_{\text{new}} = Y + (224 - X) * v_y / v_x$.

The following describes individual parts of the code that may be difficult to understand. Program uses 16-bit arithmetic to calculate the new bat position and correctly account for bounces.

To speed up the robot's operation, the program compares the horizontal and vertical speeds, and if they are equal, it jumps to calculating the result and accounting for bounces, since in this case the speeds cancel out in the formula.

```

# if vx = vy, no operations, go to finish
if
cmp r1, r0
is eq
push r3 # push vy sign in stack
ldi r3, 224
sub r3, r2
ldi r3, 0xf4
ld r3, r3
push r3 # push y to stack
ldi r3, 0x00
br finish
fi

```

The code takes the absolute value of vy so that there is no need to perform many comparisons for 7 different speeds. It saves the original sign of vy – if negative, it loads 1 into r3; otherwise, 0 remains in r3.

```

ldi r3, 0x04
and r0, r3
if
    tst r3
is ne
    neg r0
    ldi r3, 0x07
    and r3, r0
    ldi r3, 0x01
fi

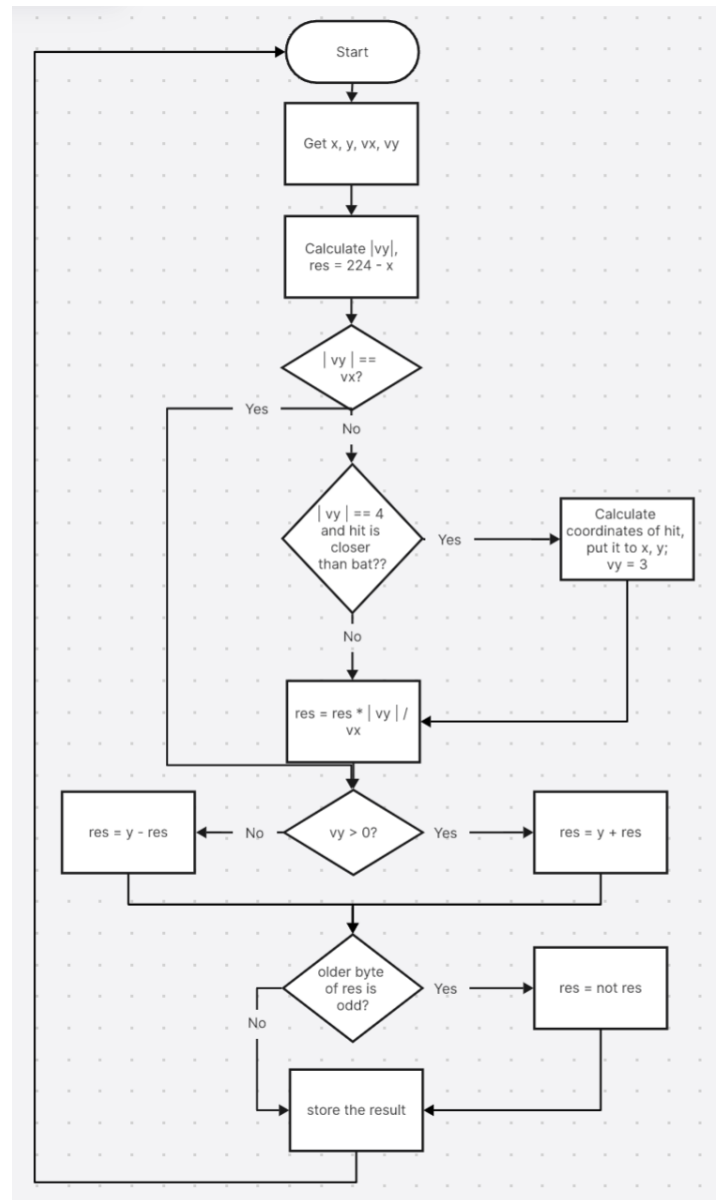
```

If the original vy = -4, the robot must take into account when calculating the trajectory that it will change to 3, not to 4. In this case, the ball's bounce coordinates are first calculated using the formula $x = x + vx * (y/4)$, $y = 0$. The code also contains a check for where the bounce will occur – if it happens after the paddle hit, then nothing needs to be changed, and the original values are preserved. Otherwise, the new coordinates are saved, the speed changes to 3, and the data is fed for further calculations.

```

if
    cmp r3, r0
is eq
    # get y from stack
    pop r3
    inc r3 # add 3 to y to round up
    inc r3
    inc r3
    move r3, r3 #clear flags to get no effect on shr
    shr r3
    move r3, r3
    shr r3
    move r3, r0
    # push vx to stack to remember it

```



```

push r1
dec r1
# calculate vx * (y / 4)
while
    tst r1
    stays gt
        add r0, r3
        if
            is cs
                ldi r1, -1
        fi
        dec r1
    wend
    add r3, r2
    if
        is cs
            ldi r1, -1
        fi
    ldi r3, 31
    add r2, r3
    if
        is cs
            ldi r1, -1
        fi
    # r1 check where the hit will be
    # if it's further than the bat, returns the original coordinates
    if
        tst r1
        is lt
            ldi r3, 0xf4
            ld r3, r3
            ldi r2, 0xf3
            ld r2, r2
            ldi r0, 4
            pop r1 # get vx from stack
        else
            # if the hit is closer than racket
            # put new coordinates
            ldi r3, 0
            pop r1 # get vx from stack
            pop r0 # get vy sign from stack
            ldi r0, 0
            push r0 # push new vy sign to stack
            ldi r0, 3
        fi
    push r3 # push new y to stack
fi

```

Multiplication in the code is performed by comparing the current vertical speed with constants from 1 to 4. To simplify the algorithm, it first determines whether the speed is even, and then sum operations are carried out (as many times as the multiplication requires).

```

# even -> r3 == 0, not even -> r3 == 1
if

```

```

        tst r3
is eq   # if vy == 2 or vy == 4
        add r2, r2
        addc r3, r3
        ldi r1, 0x04
        if # vy == 4
            cmp r0, r1
            is eq
                clr r1
                add r2, r2
                addc r1, r1
                add r3, r3
                add r1, r3
            fi
        else
            if # vy == 3
                cmp r0, r3
            is eq
                clr r3
            else
                clr r3
                add r2, r2
                addc r3, r3
                clr r0
                add r1, r2
                addc r0, r3
            fi
        fi
fi

```

Division is implemented similarly to multiplication. Since the positive horizontal speed (at which the robot's calculations take place) can be from 1 to 3, it is first compared with 2, and then with 3 (no operations are performed for a value of 1). Division by three is carried out using the operation $x = (x + 85) * 85 / 256$, since this approximately equals $x = x * 1/3$, but executes much faster than a loop subtracting three from the current number.

```

ldi r1, 0x02
if
    # if vx == 2
    cmp r1, r0
is eq
    move r3, r3
    shra r3
    shr r2
else
    ldi r1, 0x03
    if # vx == 3
# x / 3 is close to (x * 85 + 85) / 256, faster then cycle
        cmp r1, r0
        is eq
        # * 85
            move r3, r1
            move r2, r0
            shla r0

```

```

        shl r1
        shla r0
        shl r1
        add r0, r2
        addc r1, r3
        shla r0
        shl r1
        shla r0
        shl r1
        add r0, r2
        addc r1, r3
        shla r0
        shl r1
        shla r0
        shl r1
        add r0, r2
        addc r1, r3
        # + 85
        ldi r0, 0x55
        add r0, r2
        ldi r0, 0x00
        addc r0, r3
        # / 256
        move r3, r2
        clr r3
    fi
fi

```

This block is responsible for checking the sign of the vertical speed. This value is retrieved from the stack, and if the speed was negative, the sign of the previously obtained value needs to be changed. Since the arithmetic is 16-bit, the high and low byte registers are processed separately. The ball's y-coordinate is then added to the obtained value.

```

pop r3
if
    tst r3
is ne
    not r2
    not r1
    inc r2
    if
        is cs
        inc r1
    fi
fi

add r0, r2
if
is cs
    inc r1
fi

```

After all operations are completed, it is necessary to determine the number of bounces and change the value accordingly. When a negative number is obtained (checked by the most significant bit), it must be changed to a positive one, since coordinates cannot be negative. Next, the number of bounces is checked by the number of times 255 is exceeded; if the count is even, nothing is done; otherwise, the number is inverted.

```
#if r1 is neg than the res is neg
# we need to neg it because coordinate can be less than 0
if
    tst r1
is lt
    not r2
    not r1
    inc r2
    if
        is cs
            inc r1
    fi
fi
ldi r0, 1
and r0, r1
# is elder bytes odd?
# elder bytes is count of hits
# if it's odd we need to do 255 - res
if
    tst r1
is ne
    not r2
fi
```